

CA Homework 2

Friday, February 27, 2026 7:00 PM

Syed Sadeh
10303

- Question #1:

function:

addi x5, x0, 0

addi x6, x0, 0

loop_start:

bge x6, x12, loop_end

slli x7, x6, 3

add x7, x1, x7

ld x8, 0(x7)

andi x9, x8, 1

bne x9, x0, else

add x5, x5, x10

jal x0, loop_continue

else:

sub x5, x5, x8

loop_continue:

addi x6, x6, 1

jal x0, loop_start

```

    jal x0, loop_start
loop_end:
    add x10, x5, x0
    jalr x0, 0(x1)

```

• Question #2

a)

The loop contains following four instructions.

- beq (check)
- addi (x5)
- addi (x6)
- jal (loop)

- we know that the value in $x6 = 10$, so the loops runs until $x6 = 0$, basically runs 10 times.

- total instruction count:

$$10 \times 4 = 40.$$

but, beq instructions runs one more time, so,

$$\text{total inst.} = 10 \times 4 + 1 = 41$$

b)

let the values of $x5$, $x6$ be stored before loop, then.

Code:

```
for (; x6 >= 0; x6--) {  
    x5 += 2;  
}
```

- Question #3

a)

findMax:

```
ld x6, 0(x10)
```

```
addi x5, x0, 0
```

```
addi x7, x11, -1
```

array_loop:

```
bge x5, x7, Done
```

```
slli x8, x5, 3
```

```
add x8, x10, x8
```

```
ld x9, 0(x8)
```

```
ld x13, 8(x8)
```

```
bge x9, x13, skip
```

```
addi x5, x5, 1  
addi x7, x7, 1  
addi x10, x10, 4  
addi x13, x13, 4  
addi x14, x14, 4
```

bge x4, x15, skip

add x6, x13, x0

skip:

addi x5, x5, 1

jal x0, array_loop

Done:

add x10, x6, x0

jalr x0, 0(x1)

b)

findMax:

ld x6, 0(x10)

add x5, x10, x0

slli x7, x11, 3

add x7, x10, x7

loop:

bge x5, x7, end

ld x8, 0(x5)

ld x9, 8(x5)

```
ld x4, 8(x5)
bge x8, x9, skip
add x6, x9, x0
```

skip:

```
addi x5, x5, 8
jal x0, loop
```

end:

```
add x10, x6, x0
jalr x0, 0(x1)
```

c)

- Array version:

- It usually runs 8-9 instructions (9 if max is updated, 8 if not)

- Pointer version:

- It runs 6-7 instructions per loop.

- So pointer version executes lesser instructions, hence has better performance than array version.

better performance than array version.

- Question # 4:

m:

addi sp, sp, -32

sd x1, 24(sp)

sd x10, 16(sp)

sd x11, 8(sp)

sd x12, 0(sp)

ld x13, 16(sp)

ld x14, 0(sp)

add x11, x13, x14

jal x1, h

add x13, x10, x0

ld x10, 16(sp)

ld x11, 8(sp)

sub x10, x10, x11

add x11, x13, x0

jal x1, h

ld x1, 24(sp)

ld x12, 0(sp)

```
ld x12, 0(sp)
ld x11, 8(sp)
addi sp, sp, 32
jalr x0, 0(x1)
```

- Question #5:

sumDown:

```
blt x0, x10, recursive_case
addi x10, x0, 0
jalr x0, 0(x1)
```

recursive_case:

```
addi sp, sp, -16
sd x1, 8(sp)
sd x10, 0(sp)
addi x10, x10, -2
jal x1, sumDown
ld x11, 0(sp)
add x10, x11, x10
ld x1, 8(sp)
addi sp, sp, 16
jalr x0, 0(x1)
```

jalr x0, 0(x1)

Question # 6:

instruction	MemRead	Memwrite	Regwrite	MemtoReg	ALUSrc	PCsrc
add x4, x4, x5	0	0	1	0	0	0
sd x29, x2(200)	0	1	0	x	1	0
ld x8, x2(16)	1	0	1	1	1	0
addi x5, x6, -17	0	0	1	0	1	0

X = don't care, since Regwrite = 0

Question # 7:

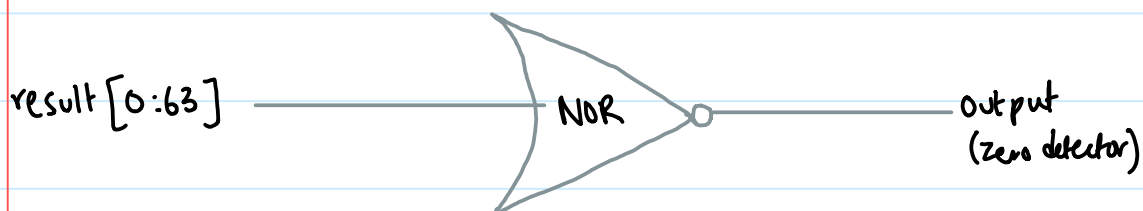
a)

When the answer = 0, so zero-detection bit = 1

Else,

zero detection bit = 0.

We can NOR all the 64-bits to get output for zero-detection bit.

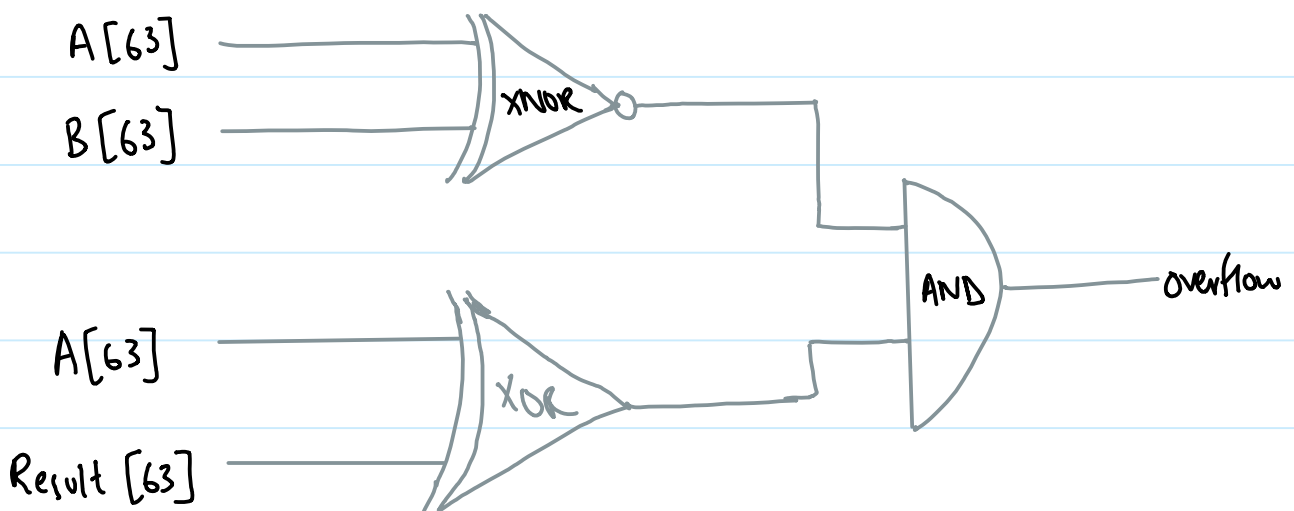


b)

b)

- for addition overflow occurs when,
 - Both operands are positive, but result is negative
 - Both operands are negative, but result is positive.
- for subtraction:
 - A is positive, B is negative, result is negative.
 - A is negative, B is positive, result is positive.

A [63]	B [63]	Result [63]	Over flow
0	0	0	0
0	0	1	1
1	1	0	1
1	1	1	0



So, overflow conditions:

for ADD:

$$\text{Overflow} = (A[63] == B[63]) \text{ AND } (A[63] != \text{Result}[63])$$

for SUB:

$$\text{Overflow} = (A[63] != B[63]) \text{ AND } (A[63] != \text{Result}[63])$$

- Question #8:

a)

The RISC-V system uses compressed 16-bit instructions that are 2 bytes in size. which means program should increment by 2 after each instruction to point to next sequential instruction. however, due to design error, the PC adder adds 4 instead of 2, so it will skip every alternating instruction in the memory. Also control flow instructions will also be missed often, that will lead to incorrect results or program will not function properly.

b, c)

even parity = 1

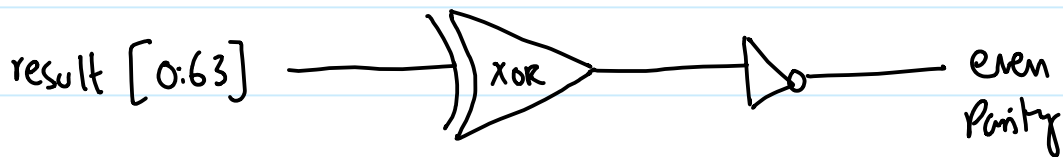
even parity = 1

- if ALU contains even number of 1's
- Parity is computed using XOR of all bits.

Property of XOR:

- XOR of all bits = 0 $\rightarrow$ even number of 1's
- XOR of all bits = 1 $\rightarrow$ odd 1's

• Hardware implementation:-



• explanation:

let P :

$$P = R_1 \oplus R_2 \oplus R_3 \oplus \dots \oplus R_{n-1}$$

then

$$\text{even parity} = \bar{P} \quad (P \text{ not})$$

- that's why we put a NOT gate after XOR.
- for odd parity, it is the direct result of XOR of

- for odd parity, it is the direct result of XOR of result of ALU.